

Cine Agent



Felipe Gonçalves Vidal, 202502208, felipevidal@discente.ufg.br
Rafael José de Souza Marques, 202502252, souza_rafael@discente.ufg.br
João Henrique Emiliano Bittencourt 202502223, joaoemiliano@discente.ufg.br
Paulo de Tarso Rezende Lôbo, 202502247, rezende.lobo@discente.ufg.br
Programação Orientada a Objetos(POO) – 2026 – Prof. Dirson S. Campos
23 de junho de 2026



Visão Geral do Projeto

- **O que é:** Um sistema de recomendação de filmes e séries que funciona através de uma interface web conversacional.
- **Objetivo:** Permitir que o usuário descreva seus gostos em linguagem natural para receber sugestões personalizadas de obras.
- **Lógica de Recomendação:** Cruza gêneros preferidos para sugerir obras similares (gêneros em comum) ou diferentes (zero gêneros em comum).
- **Stack Tecnológica:** Desenvolvido em Python, com interface no Streamlit e empacotamento via Docker.

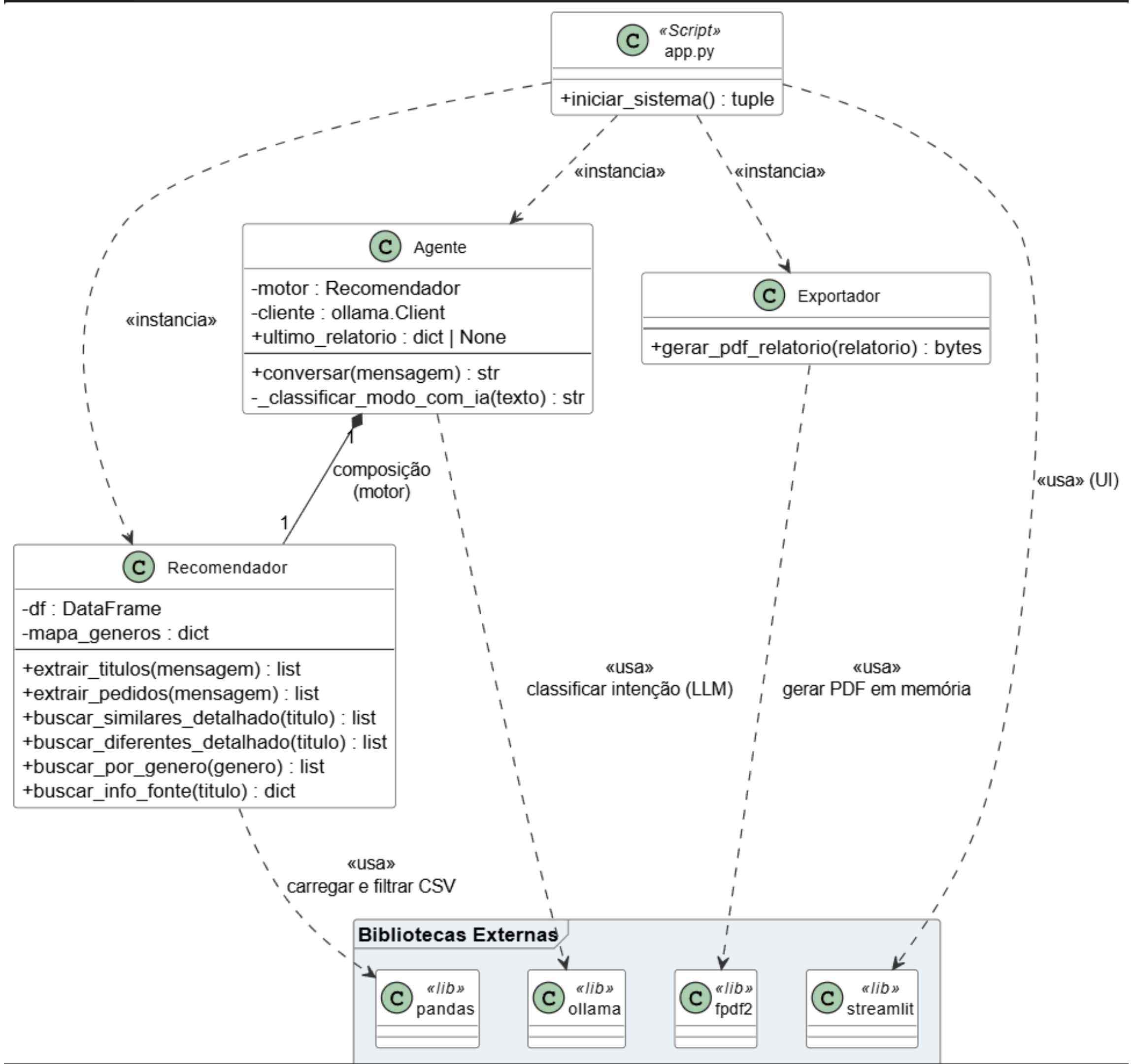
Fundamentação e Inteligência Artificial

- **Motor de Dados:** Utiliza a biblioteca Pandas para leitura, limpeza (`dropna()`) e processamento vetorizado do dataset CSV (`conteudo.csv`).
- **IA Local:** Implementa o modelo de linguagem local Qwen 2.5 (1.5B) via Ollama.
- **Classificação NLP:** A IA age exclusivamente como um classificador lógico de intenções da mensagem do usuário, evitando "alucinações" de texto.
- **Processamento:** Consegue dividir requisições compostas em múltiplas linhas de execução autônomas (ex: buscar "parecidos" e "diferentes" na mesma frase).

Arquiteturas de Classes

- **Recomendador:** Classe responsável pelo carregamento do dataset e aplicação da lógica de intersecção de conjuntos para encontrar os títulos.
- **Assistente:** Classe que configura a IA local, classifica a intenção do usuário e gerencia o histórico do chat interativo.
- **Exportador:** Classe dedicada à formatação e geração de relatórios de recomendação em formatos TXT e PDF.
- **app.py:** Ponto de entrada do Streamlit que une as classes e gerencia o estado da sessão

Arquiteturas de Classes



Conceitos de POO Aplicados

- **Classes e Objetos:** O sistema instancia objetos reais de Recomendador, Assistente e Exportador na interface principal.
- **Responsabilidade Única:** Cada classe foi desenhada para resolver exclusivamente um problema do domínio (Dados, IA e Relatórios).

Conceitos de POO Aplicados

- **Encapsulamento:** Estados e atributos importantes, como `self.df`, `self.modelo` e `self.chat`, são mantidos internos às suas respectivas classes.
- **Composição e Injeção de Dependência:** A classe `Assistente` possui uma relação de "tem-um" com o `Recomendador`, recebendo sua instância diretamente no construtor.

Exportação de Relatórios

```
class Exportador:
    def __init__(self):
        """
        Construtor. O exportador opera 100% em memória, devolvendo bytes diretamente.
        Não é necessário criar ou manter pastas de exportação.
        """
        logger.info("Exportador inicializado. Modo de operação: em memória.")

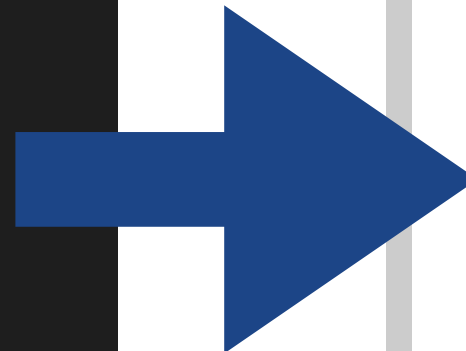
    def gerar_pdf_relatorio(self, relatorio: dict) -> bytes:
        """
        Gera um PDF estruturado a partir dos dados do CSV + sinopses do Ollama.
        """
        try:
            pdf = FPDF()
            pdf.add_page()
            pdf.set_margins(20, 20, 20)

            # Cabeçalho
            pdf.set_font("Helvetica", style="B", size=16)
            pdf.cell(0, 10, text=self._lat("Cine Agent"), ln=True, align="C")
            pdf.set_font("Helvetica", size=11)
            pdf.cell(0, 7, text=self._lat("Relatório de Recomendações"), ln=True, align="C")
            pdf.ln(4)

            # Linha separadora
            pdf.set_draw_color(180, 180, 180)
            pdf.line(20, pdf.get_y(), 190, pdf.get_y())
            pdf.ln(6)

            # Bloco da fonte
            titulo_fonte = relatorio.get("titulo_fonte", "")
            tipo_fonte = relatorio.get("tipo_fonte", "")
            genero_fonte = relatorio.get("genero_fonte", "")
            ano_fonte = relatorio.get("ano_fonte", "")

            pdf.set_font("Helvetica", style="B", size=12)
            pdf.cell(40, 8, text=self._lat("Conteúdo fonte:"))
            pdf.set_font("Helvetica", size=12)
            pdf.cell(0, 8, text=self._lat(f"{{titulo_fonte}} ({{ano_fonte}})"), ln=True)
```



Cine Agent
Relatório de Recomendações

Conteúdo fonte: Better Call Saul (2015)
Tipo: Série | Gêneros: Crime, Drama, Thriller

Conteúdos similares solicitados:

- Breaking Bad (2008)**
Série | Crime, Drama, Thriller
- Whiplash (2014)**
Filme | Drama, Music
- BoJack Horseman (2014)**
Série | Animation, Comedy, Drama
- Succession (2018)**
Série | Comedy, Drama
- Narcos (2015)**
Série | Biography, Crime, Drama

Confiabilidade e Infraestrutura

- **Catálogo de Dados:** Base CSV semi-estruturada com mais de 60 títulos.
- **Observabilidade e Segurança:** Uso de logs estruturados e blocos try/except em operações críticas.
- **Docker:** Portabilidade total e inicialização rápida de toda a infraestrutura através de um único comando (Docker Compose).
- **Escalabilidade:** A separação em camadas (Apresentação, IA, Dados, Exportação) permite adicionar novos recursos e expandir a base de dados do CSV de forma simples.

Fontes

- Repositório: <https://github.com/vvvvvdal/cine-agent>
- Python: <https://pt.wikipedia.org/wiki/Python>
- Ollama: <https://ollama.com/>
- Docker: <https://www.docker.com/>
- Pandas: <https://pandas.pydata.org/>
- FPDF: <https://pypi.org/project/fpdf2/>
- Streamlit: <https://streamlit.io/>
- TheFuzz: <https://github.com/seatgeek/thefuzz>

Cine Agent

OBRIGADO!!!

Demonstração do Código-fonte

Agora vamos para a execução!!!